Lista de exercícios – Introdução à Orientação a Objetos

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **introdução à orientação a objetos**: classe, objeto, atributos, métodos, `self`, o construtor `__init__`, acesso a atributos com `objeto.atributo`, vários objetos guardados em listas e métodos que chamam outros métodos.
- Todas as questões são de **construção de código**. Antes de implementar, separe o que o objeto **sabe** (atributos) do que ele **faz** (métodos). Desenhar o diagrama da classe, com atributos em cima e métodos embaixo, ajuda.
- Todo método recebe `self` como primeiro parâmetro, e todo atributo é acessado por meio dele: `self.saldo`, e não apenas `saldo`. Ao chamar o método, não passe o `self`: o Python faz isso sozinho.
- As regras do enunciado devem ficar **dentro dos métodos**. Um objeto deve proteger o próprio estado: por exemplo, um método de saque não deve deixar o saldo ficar negativo.
- Em cada questão, escreva também um pequeno programa principal que crie objetos e chame os métodos, reproduzindo os exemplos do enunciado. Confira se a saída obtida é igual à saída esperada.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Crie uma classe `Lampada` com os atributos `comodo` (texto), `potencia` (em watts) e `ligada`. Toda lâmpada deve ser criada desligada, ou seja, o construtor recebe apenas o cômodo e a potência.

A classe deve ter os métodos:

- `ligar()` e `desligar()`, que alteram o estado da lâmpada;
- `alternar()`, que liga a lâmpada se ela estiver desligada e desliga se estiver ligada;
- `exibir()`, que mostra o cômodo, a potência e o estado.

Exemplo de uso:

```
sala = Lampada("Sala", 60)
quarto = Lampada("Quarto", 40)
sala.ligar()
quarto.alternar()
```

```
quarto.alternar()
sala.exibir()
quarto.exibir()
```

Saída esperada:

```
Sala | 60W | ligada
Quarto | 40W | desligada
```

2. Crie uma classe `Retangulo` com os atributos `largura` e `altura` e os métodos:

- `area()`, que **devolve** a área do retângulo;
- `perimetro()`, que **devolve** o perímetro;
- `eh_quadrado()`, que devolve `True` se a largura for igual à altura;
- `redimensionar(fator)`, que multiplica a largura e a altura pelo fator informado.

Observe que os três primeiros métodos devolvem valores com `return` e não imprimem nada.

No programa principal, crie um retângulo de 4×2 e exiba sua área, seu perímetro e se ele é quadrado. Depois, redimensione-o com fator 3 e exiba novamente a área. A saída esperada é:

```
Area: 8
Perimetro: 12
Quadrado: False
Area depois: 72
```

3. Crie uma classe `ContaBancaria` com os atributos `titular` e `saldo`. O saldo inicial deve ser zero. A classe deve ter os métodos:

- `depositar(valor)`: soma o valor ao saldo, desde que ele seja positivo; caso contrário, exibe `Valor invalido`;
- `sacar(valor)`: subtrai o valor do saldo, desde que haja saldo suficiente; caso contrário, exibe `Saldo insuficiente`. O método deve devolver `True` se o saque foi realizado e `False` caso contrário;
- `exibir()`: mostra o titular e o saldo com duas casas decimais.

Exemplo de uso:

```
conta = ContaBancaria("Ana")
conta.depositar(100)
conta.depositar(-5)
conta.sacar(30)
conta.sacar(500)
conta.exibir()
```

Saída esperada:

```
Valor invalido
Saldo insuficiente
Ana | Saldo: R$ 70.00
```

4. Crie uma classe `Ponto` com os atributos `x` e `y` e os métodos:

- `exibir()`: mostra o ponto no formato `(x, y)`;
- `mover(dx, dy)`: soma `dx` ao `x` e `dy` ao `y`;
- `distancia_origem()`: devolve a distância do ponto até a origem, calculada como $|x| + |y|$;
- `quadrante()`: devolve o número do quadrante (1, 2, 3 ou 4) em que o ponto está, ou 0 se ele estiver sobre um dos eixos.

Lembre-se: no 1º quadrante, $x > 0$ e $y > 0$; no 2º, $x < 0$ e $y > 0$; no 3º, $x < 0$ e $y < 0$; no 4º, $x > 0$ e $y < 0$.

No programa principal, crie os pontos $(2, 3)$ e $(-4, 1)$. Mova o segundo ponto com `mover(1, -6)` e exiba, para cada ponto, suas coordenadas, sua distância até a origem e seu quadrante. O segundo ponto deve terminar em $(-3, -5)$, com distância 8, no 3º quadrante.

5. Crie uma classe `Item` para representar os itens que um personagem carrega. Cada item tem `nome`, `quantidade` e `limite` (a quantidade máxima que pode ser carregada). A classe deve ter os métodos:

- `exibir()`: mostra o nome, a quantidade e o limite;
- `adicionar(qtd)`: soma `qtd` à quantidade, sem ultrapassar o limite. Se a soma ultrapassar o limite, a quantidade deve ficar igual ao limite, e o método deve exibir quantas unidades foram descartadas;
- `usar()`: diminui 1 da quantidade, se ainda houver itens, e exibe `Usou <nome>`; caso contrário, exibe `Nao ha <nome>`.

Exemplo de uso:

```
pocao = Item("Pocao", 2, 5)
pocao.usar()
pocao.adicionar(6)
pocao.exibir()
```

Saída esperada:

```
Usou Pocao
2 unidade(s) de Pocao descartada(s)
Pocao: 5/5
```

6. Crie uma classe `Aluno` com os atributos `nome` e `notas`, sendo `notas` uma lista que começa vazia. A classe deve ter os métodos:

- `adicionar_nota(nota)`: acrescenta a nota à lista, desde que ela esteja entre 0 e 10;
- `media()`: devolve a média das notas, ou 0 se não houver notas;
- `situacao()`: devolve `"Aprovado"` se a média for maior ou igual a 6 e `"Reprovado"` caso contrário. Esse método deve **chamar o método** `media()` do próprio objeto;
- `exibir()`: mostra o nome, as notas, a média com uma casa decimal e a situação.

No programa principal, crie dois alunos, adicione três notas a cada um (incluindo pelo menos uma nota inválida, que deve ser ignorada) e exiba os dois.

7. Crie uma classe `Produto` com os atributos `nome`, `preco` e `estoque` e os métodos:

- `vender(qtd)`: se houver estoque suficiente, reduz o estoque e devolve o valor da venda; caso contrário, devolve 0;
- `repor(qtd)`: aumenta o estoque;
- `aplicar_desconto(percentual)`: reduz o preço no percentual informado, desde que ele esteja entre 0 e 50;
- `exibir()`: mostra o nome, o preço com duas casas decimais e o estoque.

No programa principal, crie uma **lista** com pelo menos três produtos. Em seguida, percorra a lista com `for` para:

a) vender 2 unidades de cada produto, acumulando o total arrecadado;
b) aplicar 10% de desconto nos produtos com estoque maior que 10;
c) exibir todos os produtos.

Ao final, exiba o total arrecadado e o nome do produto com o maior estoque.

8. Crie uma classe `Cronometro` que represente um tempo em horas, minutos e segundos. O construtor recebe os três valores. A classe deve ter os métodos:

- `total_segundos()`: devolve o tempo total convertido em segundos;
- `avancar(segundos)`: avança o cronômetro a quantidade de segundos informada, mantendo os minutos e os segundos sempre entre 0 e 59;
- `exibir()`: mostra o tempo no formato `hh:mm:ss`, sempre com dois dígitos em cada parte.

Por exemplo:

```
c = Cronometro(1, 58, 30)
c.avancar(95)
c.exibir()
print(c.total_segundos())
```

deve exibir:

```
02:00:05
7205
```

Dica: uma estratégia é converter tudo para segundos, somar e depois recalcular horas, minutos e segundos com `//` e `%`. Para exibir com dois dígitos, use `f"{valor:02d}"`.

9. Crie uma classe `Personagem` com os atributos `nome`, `vida`, `ataque` e `defesa` e os métodos:

- `esta_vivo()`: devolve `True` se a vida for maior que zero;
- `receber_dano(valor)`: calcula `dano_final = valor - defesa`. Se o dano final for menor que 1, ele deve ser ajustado para 1. A vida nunca pode ficar menor que 0, e um personagem derrotado não pode receber mais dano;
- `atacar(outro)`: causa dano em outro personagem, chamando o `receber_dano` do outro objeto com o seu próprio ataque. Um personagem derrotado não pode atacar;
- `curar(valor)`: aumenta a vida, sem ultrapassar a vida máxima do personagem (a vida com que ele foi criado). Um personagem derrotado não pode ser curado.

Observe que, para implementar `curar`, o objeto precisa guardar a vida máxima em um atributo próprio no construtor.

No programa principal, crie dois personagens e simule uma batalha em turnos com um laço `while`: enquanto os dois estiverem vivos, o primeiro ataca o segundo e, se o segundo continuar vivo, ele ataca o primeiro. Ao final, exiba quantos turnos foram necessários e o nome do vencedor.

10. Uma empresa de aluguel de bicicletas deseja controlar seus aluguéis. Crie uma classe `Bicicleta` com os atributos `codigo`, `valor_hora`, `disponivel` (inicialmente `True`), `horas_alugada` (inicialmente 0) e `faturamento` (inicialmente 0).

A classe deve ter os métodos:

- `alugar()`: se a bicicleta estiver disponível, marca-a como indisponível e devolve `True`; caso contrário, devolve `False`;
- `devolver(horas)`: se a bicicleta estiver alugada, marca-a como disponível, soma as horas ao total, calcula o valor do aluguel, soma esse valor ao faturamento e devolve o valor cobrado. Aluguéis com mais de 3 horas recebem 20% de desconto sobre o valor total. Se a bicicleta não estiver alugada, o método deve devolver 0;
- `exibir()`: mostra o código, a situação (disponível ou alugada), o total de horas e o faturamento.

No programa principal, crie uma lista com três bicicletas de valores diferentes e leia uma quantidade indeterminada de operações. Cada operação é formada pelo tipo (`alugar` ou `devolver`) e pelo código da bicicleta; no caso de uma devolução, também é lida a quantidade de horas. A leitura termina quando o tipo for `fim`.

Para cada operação, localize a bicicleta na lista pelo código e exiba uma mensagem informando se a operação foi realizada, recusada ou se a bicicleta não existe. No caso de uma devolução, exiba também o valor cobrado.

Ao final, exiba todas as bicicletas, o faturamento total da empresa e o código da bicicleta que mais faturou.